
COMP 1023 Sample Exam - HKUST

Date: — —, — (—)

Time Allowed: — hours, —:— to —:— pm

- Instructions:
1. This is a closed-book, closed-notes examination.
 2. There are 7 questions on **31** pages (including this cover page, honor code, and 4 blank pages at the end).
 3. Write your answers in the space provided in black/blue ink. *NO pencils please, otherwise you are not allowed to appeal for any grading disagreements.*
 4. All programming codes in your answers must be written in the Python version as taught in the class.
 5. For programming questions, unless otherwise stated, you are **NOT** allowed to define additional classes, helper functions (including inner functions) and use global variables, nor any library functions not mentioned in the questions.
 6. Type hinting does not need to be included in your code.
 7. No electronic devices are allowed, except for an HKEAA approved calculator.

Student Name	SOLUTIONS
Student ID	
Venue & Seat Number	

For T.A.

Use Only

Problem	Topic	Score
1	True/False Questions	
2	SQ: Python Operations	
3	SQ: Branching	
4	SQ: Looping	
5	SQ: Collections (Lists)	
6	SQ: Functions	
7	LQ: Book Management System	
	Total	/ 100

SQ: Short Question; LQ: Long Question

Disclaimer

This sample paper is intended to help you understand the format of the upcoming exams and the types of questions you may encounter. Please note that it may not cover all possible question types, as we may include additional formats. Furthermore, the number of short and long questions, as well as the difficulty and length of the exam questions, may vary from those presented in this sample.

We strongly encourage you to thoroughly review all lecture notes, lab exercises, tutorial materials, and self-tests, focusing on understanding the concepts rather than simply memorizing facts or relying too heavily on practice questions.

The HKUST Academic Honor Code

Honesty and integrity are central to the academic work of HKUST. Students of the University must observe and uphold the highest standards of academic integrity and honesty in all the work they do throughout their program of study.

As members of the University community, students have the responsibility to help maintain the academic reputation of HKUST in its academic endeavors.

Sanctions will be imposed if students are found to have violated the regulations governing academic integrity and honesty.

Declaration of Academic Integrity

I confirm that I have answered the questions using only materials specifically approved for use in this examination, that all the answers are my own work, and that I have not received any assistance during the examination.

Signature: _____

Problem 1 True/False Questions

Indicate whether the following statements are true or false by putting T or F in the given table. You get 1 point for each correct answer.

- (a) In Python, the statement `name = input("Enter a value: ")` will store the user's input as an integer if they type a number.
- (b) A string in Python is an example of a mutable data type, meaning you can change individual characters after it's created (e.g., `my_string[0] = 'A'`).
- (c) In Python, when using the expression `a and b`, if `a` evaluates to `False` in a boolean context, then `b` is never evaluated.
- (d) In an `if...elif...else` block, once one condition is met, the rest of the conditions are skipped.
- (e) A while loop will always execute its code block **at least once**, even if the condition is initially `False`.
- (f) The `list.append()` method is used to add a single new element to the end of a list.
- (g) When defining a function in Python, parameters with default values must always come before parameters without default values in the parameter list.
- (h) A function in Python can return multiple values by separating them with commas, which are returned as a tuple.
- (i) The `is` operator checks if two variables have the same value, while the `==` operator checks if they reference to the same object in memory.
- (j) In Python, if you assign one list to another using the assignment operator (`=`), modifying the new list will also change the original list.

Question	(a)	(b)	(c)	(d)	(e)	(f)	(g)	(h)	(i)	(j)
Answer										

Solution:

Question	(a)	(b)	(c)	(d)	(e)	(f)	(g)	(h)	(i)	(j)
Answer	F	F	T	T	F	T	F	T	F	T

- (a) False. The `input()` function always returns a string. Even if the user types “123”, it is stored as the string “123”.
- (b) False. Strings are immutable. Trying to change a character directly with `my_string[0] = 'A'` will result in a `TypeError`.
- (c) True. This demonstrates short-circuit evaluation. When `a` is falsy in `a and b`, Python doesn’t evaluate `b` because the overall result must be False regardless of `b`’s value.
- (d) True. The `if...elif...else` structure is evaluated from top to bottom. The first condition that evaluates to True has its block executed, and then the entire structure is exited.
- (e) False. A while loop checks its condition before each iteration. If the condition is False at the very start, the code block will never run.
- (f) True. The `list.append(x)` method adds the object `x` to the end of the list.
- (g) False. In Python, parameters with default values (default parameters) can come after parameters without default values, but not before.
- (h) True. For example, `return a, b` will return the tuple `(a, b)`.
- (i) False. This is backwards. The `==` operator checks for value equality. The `is` operator checks for identity, meaning it checks if two variables reference to the exact same object in memory.
- (j) True. This happens because the assignment operator creates a reference to the same list object, not a copy. To avoid this, you can use `.copy()` or `list()` to create a new list.

Problem 2 Python Operations

What is the final value of the variable **result** after each of the following code blocks executes? Show your work by evaluating the expressions step-by-step.

(a) `x = 5`

`y = 3`

`result = x % y + 2 * 3 > y ** 2`

(b) `my_list = [10, 20, 30]`

`result = 25 not in my_list and not (10 > 5 or 5 == 3)`

(c) `a = [1, 2, 3]`

`c = a`

`b = [1, 2, 3]`

`result = list(a) is c or a == b and a is not b`

Solution:

- (a)
- `x % y` is `5 % 3 = 2`
 - `2 * 3 = 6`
 - `y ** 2` is `3 ** 2 = 9`
 - Left side becomes `2 + 6 = 8`
 - Final expression: `8 > 9`
 - Final result: `False`
- (b)
- `25 not in my_list` is `True` (because 25 is not in `[10, 20, 30]`)
 - Evaluate the parenthesis: `(10 > 5 or 5 == 3)` is `(True or False) = True`
 - `not (True) = False`
 - Final expression: `True and False`
 - Final result: `False`
- (c)
- Due to operator precedence: `and` has higher precedence than `or`, so it's equivalent to:
`result = (list(a) is c) or (a == b and a is not b)`
 - Evaluate `list(a) is c`
 - `list(a)` creates a new list with the same elements as `a`
 - `c` is the original list `a`
 - `list(a) is c` compares if they are the same object in memory
 - Result: `False` (they are different objects)
 - Since first part is `False`, evaluate the `and` expression
 - `a == b and a is not b`
 - Evaluate `a == b`
 - Both lists have the same elements `[1, 2, 3]`
 - Result: `True` (value equality)
 - Evaluate `a is not b`
 - `a` and `b` are different objects in memory
 - Evaluate the `and` expression
 - `True and True = True`
 - Evaluate the final `or` expression
 - `False or True = True`
 - Final result: `True`

Problem 3 Branching

Predict the output of the following code blocks. If no output is produced, write “No output”.

- (a) `temperature = 85`
`if temperature > 90:`
 `print("Hot")`
`elif temperature > 70:`
 `print("Warm")`
`elif temperature > 50:`
 `print("Cool")`
`else:`
 `print("Cold")`
- (b) `x = 10`
`y = 5`
`if x + y > 20:`
 `print("A")`
`if x * 2 == 20:`
 `print("B")`
`if y - x < 0:`
 `print("C")`
- (c) `is_weekend = True`
`is_sunny = False`
`if is_weekend and is_sunny:`
 `print("Go to the beach")`
`elif is_weekend and not is_sunny:`
 `print("Watch a movie")`
`elif not is_weekend:`
 `print("Go to work")`
`else:`
 `print("Read a book")`
- (d) `value = 5`
`match value:`
 `case 1 | 2 | 3:`
 `print("Small")`
 `case 4 | 5 | 6:`
 `print("Medium")`
 `case 7 | 8 | 9:`
 `print("Large")`
 `case _:`
 `print("Unknown")`

Solution:

- (a)
- The first condition $85 > 90$ is **False**.
 - The second condition $85 > 70$ is **True**.
 - The first **True** condition in an if...elif...else chain is executed, and the rest are skipped.
 - Output: Warm
- (b)
- $x + y > 20$ is $15 > 20 \rightarrow$ **False**. “A” is not printed.
 - $x * 2 == 20$ is $20 == 20 \rightarrow$ **True**. “B” is printed.
 - $y - x < 0$ is $-5 < 0 \rightarrow$ **True**. “C” is printed.
 - Output:
B
C
- (c)
- `is_weekend` is **True**.
 - `is_sunny` is **False**.
 - Condition 1: **True and False** $\rightarrow$ **False**.
 - Condition 2: **True and not False** $\rightarrow$ **True and True** $\rightarrow$ **True**.
 - This condition is met, so the corresponding block executes.
 - Output: Watch a movie
- (d)
- The value 5 matches the second case $4 \mid 5 \mid 6$, so “Medium” is printed.
 - Output: Medium

Problem 4 Looping

Predict the output of the following code blocks.

- (a) `total = 0`
 `for i in range(2, 8, 2):`
 `total = total + i`
 `print(total)`
- (b) `count = 5`
 `while count > 0:`
 `print(count, end=" ")`
 `count -= 2`
 `print("Done!")`
- (c) `for num in [3, 7, 2, 9, 5]:`
 `if num % 4 == 0:`
 `print("Found multiple of 4")`
 `break`
 `print(num)`
 `else:`
 `print("No multiples of 4 found")`
- (d) `x = 5`
 `while x > 0:`
 `x -= 1`
 `if x == 2:`
 `continue`
 `print(x, end=" ")`

Solution:

- (a)
- `range(2, 8, 2)` generates: 2, 4, 6
 - Loop iterations:
 - `i = 2: total = 0 + 2 = 2`
 - `i = 4: total = 2 + 4 = 6`
 - `i = 6: total = 6 + 6 = 12`
 - Output: 12
- (b)
- Initial `count = 5`
 - Iteration 1: Print 5, `count` becomes 3
 - Iteration 2: Print 3, `count` becomes 1
 - Iteration 3: Print 1, `count` becomes -1
 - Condition `-1 > 0` is False, loop exits
 - Print “Done!”
 - Output: `5_3_1_Done!`
- (c)
- `num = 3: 3 % 4 ≠ 0`, print 3
 - `num = 7: 7 % 4 ≠ 0`, print 7
 - `num = 2: 2 % 4 ≠ 0`, print 2
 - `num = 9: 9 % 4 ≠ 0`, print 9
 - `num = 5: 5 % 4 ≠ 0`, print 5
 - Loop completes normally without hitting `break`
 - The `else` clause associated with the loop executes
 - Output:
 - 3
 - 7
 - 2
 - 9
 - 5
 - No multiples of 4 found

- (d)
- Initial state: $x = 5$
 - Iteration 1:
 - Condition $x > 0$ ($5 > 0$) is **True**, enter loop
 - $x -= 1 \rightarrow x = 4$
 - Check **if** $x == 2 \rightarrow 4 == 2$ is **False**
 - Execute **print**(x , **end**=" ") $\rightarrow$ Output: 4
 - Iteration 2:
 - Condition $x > 0$ ($4 > 0$) is **True**, enter loop
 - $x -= 1 \rightarrow x = 3$
 - Check **if** $x == 2 \rightarrow 3 == 2$ is **False**
 - Execute **print**(x , **end**=" ") $\rightarrow$ Output: 4 3
 - Iteration 3:
 - Condition $x > 0$ ($3 > 0$) is **True**, enter loop
 - $x -= 1 \rightarrow x = 2$
 - Check **if** $x == 2 \rightarrow 2 == 2$ is **True**
 - Execute **continue** $\rightarrow$ skip remaining code in loop, jump to next iteration
 - No print statement executed
 - Iteration 4:
 - Condition $x > 0$ ($2 > 0$) is **True**, enter loop
 - $x -= 1 \rightarrow x = 1$
 - Check **if** $x == 2 \rightarrow 1 == 2$ is **False**
 - Execute **print**(x , **end**=" ") $\rightarrow$ Output: 4 3 1
 - Iteration 5:
 - Condition $x > 0$ ($1 > 0$) is **True**, enter loop
 - $x -= 1 \rightarrow x = 0$
 - Check **if** $x == 2 \rightarrow 0 == 2$ is **False**
 - Execute **print**(x , **end**=" ") $\rightarrow$ Output: 4 3 1 0
 - Iteration 6:
 - Condition $x > 0$ ($0 > 0$) is **False**, exit loop
 - Final output: 4 3 1 0

Problem 5 Collections (Lists)

Predict the output of the following code blocks. If an error occurs, state what the error is.

- (a) `my_list = [1, 3, 5]`
`my_list.append(7)`
`my_list.insert(1, 2)`
`my_list.remove(3)`
`print(my_list)`
- (b) `numbers = [10, 20, 30, 40, 50, 60]`
`result = numbers[1:4] + numbers[-2:]`
`print(result)`
- (c) `list_a = [5, 1, 4]`
`list_b = list_a`
`list_c = list_a.copy()`

`list_a.sort()`
`list_b.append(10)`
`list_c.extend([2, 8])`

`print("A:", list_a)`
`print("B:", list_b)`
`print("C:", list_c)`

Solution:

- (a)
 - Start: [1, 3, 5]
 - `append(7)`: [1, 3, 5, 7]
 - `insert(1, 2)`: [1, 2, 3, 5, 7]
 - `remove(3)`: removes first occurrence of 3 → [1, 2, 5, 7]
 - Output: [1, 2, 5, 7]
- (b)
 - `numbers[1:4]` gets elements at indices 1, 2, 3 → [20, 30, 40]
 - `numbers[-2:]` gets last 2 elements → [50, 60]
 - [20, 30, 40] + [50, 60] = [20, 30, 40, 50, 60]
 - Output: [20, 30, 40, 50, 60]
- (c)
 - `list_b = list_a` → `list_b` references the same object as `list_a`
 - `list_c = list_a.copy()` → `list_c` is a new, independent copy
 - `list_a.sort()` → sorts the original list that both `list_a` and `list_b` reference → [1, 4, 5]
 - `list_b.append(10)` → modifies the same list → [1, 4, 5, 10]
 - `list_c.extend([2, 8])` → modifies the independent copy → [5, 1, 4, 2, 8]
 - Output:
 - A: [1, 4, 5, 10]
 - B: [1, 4, 5, 10]
 - C: [5, 1, 4, 2, 8]

Problem 6 Functions

Consider the following function:

```
def mystery(a, b=1, mode="add"):
    if mode == "add":
        return a + b
    elif mode == "multiply":
        return a * b
    elif mode == "power":
        return a ** b
    else:
        return "Invalid mode"
```

(a) What is the output of each function call below?

- (i) `print(mystery(5, 3))`
- (ii) `print(mystery(4, mode="multiply"))`
- (iii) `print(mystery(2, 4, "power"))`
- (iv) `print(mystery(7))`

(b) What does this function call return and why?

```
result = mystery(10, mode="add", b=2)
print(result)
```

(c) Based on all the examples above, describe in one sentence what this function does and identify the purpose of each parameter.

Solution:

- (a) (i) `print(mystery(5, 3))` *# Output: 8*
 `mystery(5, 3)` uses default `mode="add"`, so it returns $5 + 3 = 8$
- (ii) `print(mystery(4, mode="multiply"))` *# Output: 4*
 `mystery(4, mode="multiply")` uses `b=1` (default), so it returns $4 \times 1 = 4$
- (iii) `print(mystery(2, 4, "power"))` *# Output: 16*
 `mystery(2, 4, "power")` returns $2 ** 4 = 16$
- (iv) `print(mystery(7))` *# Output: 8*
 `mystery(7)` uses `b=1` (default) and `mode="add"` (default), so it returns $7 + 1 = 8$
- (b) `result = mystery(10, mode="add", b=2)` *# Output: 12*

Explanation: This returns 12 because it uses keyword arguments to specify `a=10`, `b=2`, and `mode="add"`, so it performs $10 + 2 = 12$. The order of keyword arguments doesn't matter as long as they come after positional arguments.

- (c) Function Purpose: This function performs different mathematical operations on two numbers based on the specified mode.

Parameter purposes:

- `a`: The first number (required)
- `b`: The second number (optional, defaults to 1)
- `mode`: The operation to perform - "add", "multiply", or "power" (optional, defaults to "add")

Problem 7 Book Management System

You are developing a book management system for a small library. The system should allow librarians to manage their book collection and help members find books. The program has 7 functions, with 2 functions (`initialize_library` and `main()`) already implemented for you. You need to implement the remaining 5 functions: (`add_books`, `borrow_book`, `return_book`, `search_books`, and `get_library_status`). Below are the details of the functions that you are required to implement.

```
def initialize_library(): # GIVEN
    """
    Initializes the library with sample books using lists
    Returns books list and borrowed list
    """
    book_titles = ["Python Programming", "Data Structures", "Algorithms", \
                   "Machine Learning", "Database Systems"]
    book_copies = [3, 2, 1, 2, 2] # 3 Python Programming, ..., 2 Database Systems
    book_borrow_count = [0, 0, 0, 0, 0] # All the books haven't been borrowed

    books_list = [book_titles, book_copies, book_borrow_count]
    borrowed_list = [] # Will store [title, member_name] pairs

    return books_list, borrowed_list

def add_book(books_list, new_book, quantity=1):
    """
    Adds a new book to the library collection or updates quantity if book exists

    Requirements:
    - books_list is a list of lists: [titles, copies, borrow_count]
    - Check if book already exists
    - If book exists: increase the copies count by quantity
    - If book doesn't exist: add new book with given quantity and initialize
      borrow_count to 0
    - Validate inputs: new_book must be non-empty string, quantity must be
      positive integer
    - Return updated books_list and status message string

    Parameters:
    books_list: list - [list_of_titles, list_of_copies, list_of_borrow_counts]
    new_book: str - title of book to add
    quantity: int - number of copies to add (default=1)

    Returns:
    tuple: (updated_books_list, status_message)
    """
    # TODO (a): Implement book addition logic
    # Your code here!
```

```

def borrow_book(books_list, borrowed_list, book_title, member_name):
    """
    Allows a member to borrow a book if it's available

    Requirements:
    - books_list is a list of lists: [titles, copies, borrow_count]
    - borrowed_list is a list of [book_title, member_name] pairs
    - Search for book_title in books_list
    - If book found and copies available:
        - Decrease copies count by 1
        - Increase borrow_count for that book by 1
        - Add [book_title, member_name] to borrowed_list
        - Return success status True with success message
    - If book not found or no copies available:
        - Return success status False with appropriate error message
    - Validate inputs: book_title and member_name must be non-empty strings

    Parameters:
    books_list: list - [list_of_titles, list_of_copies, list_of_borrow_counts]
    borrowed_list: list - list of [book_title, member_name] pairs
    book_title: str - title of book to borrow
    member_name: str - name of member borrowing the book

    Returns:
    tuple: (updated_books_list, updated_borrowed_list, success_status, message)
    """
    # TODO (b): Implement book borrowing logic
    # Your code here!

def return_book(books_list, borrowed_list, book_title):
    """
    Processes book returns and moves book from borrowed back to available

    Requirements:
    - books_list is a list of lists: [titles, copies, borrow_count]
    - borrowed_list is a list of [book_title, member_name] pairs
    - Search for book_title in borrowed_list
    - If book found in borrowed_list:
        - Remove the book from borrowed_list
        - Increase copies count in books_list by 1
        - Return success status True with success message
    - If book not found in borrowed_list:
        - Return success status False with error message
    - Validate input: book_title must be non-empty string

    Parameters:
    books_list: list - [list_of_titles, list_of_copies, list_of_borrow_counts]
    borrowed_list: list - list of [book_title, member_name] pairs
    book_title: str - title of book to return

```

```

Returns:
tuple: (updated_books_list, updated_borrowed_list, success_status, message)
"""

# TODO (c): Implement book return logic
# Your code here!

def search_books(books_list, search_term):
    """
    Searches for books by title using partial matching

    Requirements:
    - books_list is a list of lists: [titles, copies, borrow_count]
    - Search for search_term in all book titles (partial match)
    - Return list of indices where search_term appears in book titles
    - If no matches found, return empty list
    - If search_term is empty, return empty list

    Parameters:
    books_list: list - [list_of_titles, list_of_copies, list_of_borrow_counts]
    search_term: str - term to search for in book titles

    Returns:
    list: indices of matching books in the books_list
    """
    # TODO (d): Implement search functionality
    # Your code here!

def get_library_stats(books_list, borrowed_list):
    """
    Calculates and returns library usage statistics

    Requirements:
    - books_list is a list of lists: [titles, copies, borrow_count]
    - borrowed_list is a list of [book_title, member_name] pairs
    - Calculate:
      - total_available: sum of all copies in books_list
      - total_borrowed: length of borrowed_list
      - total_books: total_available + total_borrowed
      - most_popular: book title with highest borrow_count
      - utilization: (total_borrowed / total_books) * 100
    - If no books have been borrowed, most_popular should be "No borrows yet"
    - If total_books is 0, utilization should be 0

    Parameters:
    books_list: list - [list_of_titles, list_of_copies, list_of_borrow_counts]
    borrowed_list: list - list of [book_title, member_name] pairs

```

```

Returns:
list: [total_available, total_borrowed, total_books, most_popular, utilization]
"""

# TODO (e): Implement statistics calculation
# Your code here!

def main():
    # Initialize
    books, borrowed = initialize_library()
    print("1. Library initialized")

    # Test key outputs users need to handle
    print("\n2. ADD BOOK outputs:")
    books, msg = add_book(books, "Deep Learning", 2)
    print(f"    Success: {msg}")
    books, msg = add_book(books, "Python Programming", 1)
    print(f"    Success: {msg}")
    books, msg = add_book(books, "", 1) # Error
    print(f"    Error: {msg}")

    print("\n3. BORROW BOOK outputs:")
    books, borrowed, success, msg = borrow_book(books, borrowed,
                                                "Python Programming", "Alice")
    print(f"    Success: {success}, Message: {msg}")
    books, borrowed, success, msg = borrow_book(books, borrowed, "Unknown Book", "Bob")
    print(f"    Success: {success}, Message: {msg}")

    print("\n4. RETURN BOOK outputs:")
    books, borrowed, success, msg = return_book(books, borrowed, "Python Programming")
    print(f"    Success: {success}, Message: {msg}")
    books, borrowed, success, msg = return_book(books, borrowed, "Never Borrowed")
    print(f"    Success: {success}, Message: {msg}")

    print("\n5. SEARCH outputs:")
    results = search_books(books, "Python")
    print(f"    Found {len(results)} books")
    results = search_books(books, "xyz")
    print(f"    Found {len(results)} books (no results)")

    print("\n6. STATISTICS outputs:")
    stats = get_library_stats(books, borrowed)
    print(f"    Stats list: {stats}")
    print(f"    Available: {stats[0]}, Borrowed: {stats[1]}")

if __name__ == "__main__":
    main()

```

Sample output:

1. Library initialized
 2. ADD BOOK outputs:
Success: Added new book: 'Deep Learning' with 2 copies
Success: Added 1 more copies of 'Python Programming'. Total: 4
Error: Invalid book title or quantity
 3. BORROW BOOK outputs:
Success: True, Message: Book 'Python Programming' borrowed successfully by Alice
Success: False, Message: Book 'Unknown Book' not found in library
 4. RETURN BOOK outputs:
Success: True, Message: Book 'Python Programming' returned successfully
Success: False, Message: Book 'Never Borrowed' was not found in borrowed list
 5. SEARCH outputs:
Found 1 books
Found 0 books (no results)
 6. STATISTICS outputs:
Stats list: [13, 0, 13, 'Python Programming', 0.0]
Available: 13, Borrowed: 0
-
- (a) Implement the `add_book` function according to the given description.
 - (b) Implement the `borrow_book` function according to the given description.
 - (c) Implement the `return_book` function according to the given description.
 - (d) Implement the `search_books` function according to the given description.
 - (e) Implement the `get_library_stats` function according to the given description.

Solution:

```
(a) def add_book(books_list, new_book, quantity=1):
    # Input validation
    if not new_book or quantity <= 0:
        return books_list, "Invalid book title or quantity"

    titles, copies, borrow_count = books_list

    # Check if book already exists
    book_found = False
    for i in range(len(titles)):
        if titles[i] == new_book:
            copies[i] += quantity
            book_found = True
            return books_list, \
                f"Added {quantity} more copies of '{titles[i]}'. Total: {copies[i]}"

    # Add new book
    titles.append(new_book)
    copies.append(quantity)
    borrow_count.append(0)

    return books_list, f"Added new book: '{new_book}' with {quantity} copies"
```

```

(b) def borrow_book(books_list, borrowed_list, book_title, member_name):
    # Input validation
    if not book_title or not member_name:
        return books_list, borrowed_list, False, "Invalid book title or member name"

    titles, copies, borrow_count = books_list

    # Search for the book
    for i in range(len(titles)):
        if titles[i] == book_title:
            if copies[i] > 0:
                # Book available, process borrowing
                copies[i] -= 1
                borrow_count[i] += 1

                # Add to borrowed list
                borrowed_list.append([titles[i], member_name])

            return books_list, borrowed_list, True, \
                f"Book '{titles[i]}' borrowed successfully by {member_name}"
        else:
            return books_list, borrowed_list, False, \
                f"Sorry, '{titles[i]}' is currently unavailable"

    return books_list, borrowed_list, False, \
        f"Book '{book_title}' not found in library"

```

```

(c) def return_book(books_list, borrowed_list, book_title):
    if not book_title:
        return books_list, borrowed_list, False, "Invalid book title"

    titles, copies, borrow_count = books_list

    # Find book in borrowed list
    for i in range(len(borrowed_list)):
        if borrowed_list[i][0] == book_title:
            # Remove from borrowed list
            returned_book_title = borrowed_list[i][0]
            borrowed_list.pop(i)

            # Add back to available books
            for j in range(len(titles)):
                if titles[j] == book_title:
                    copies[j] += 1
                    return books_list, borrowed_list, True, \
                        f"Book '{titles[j]}' returned successfully"

    return books_list, borrowed_list, False, \
        f"Book '{book_title}' was not found in borrowed list"

```



```
(d) def search_books(books_list, search_term):  
    if not search_term:  
        return []  
  
    titles, copies, borrow_count = books_list  
    matches = []  
    search_term = search_term  
  
    for i in range(len(titles)):  
        if search_term in titles[i]:  
            matches.append(i)  
  
    return matches
```

```

(e) def get_library_stats(books_list, borrowed_list):
    titles, copies, borrow_count = books_list

    total_available = 0
    for count in copies:
        total_available += count

    total_borrowed = len(borrowed_list)
    total_books = total_available + total_borrowed

    # Find most popular book
    most_popular_index = -1
    max_borrows = 0

    for i in range(len(borrow_count)):
        if borrow_count[i] > max_borrows:
            max_borrows = borrow_count[i]
            most_popular_index = i

    most_popular = "No borrows yet"
    if most_popular_index >= 0 and max_borrows > 0:
        most_popular = titles[most_popular_index]

    # Calculate utilization percentage
    utilization = 0
    if total_books > 0:
        utilization = (total_borrowed / total_books) * 100

    # Return as list
    stats = [
        total_available,
        total_borrowed,
        total_books,
        most_popular,
        utilization
    ]

    return stats

----- END OF PAPER -----

```

Appendix:

Operator Precedence from High (Top) to Low (Down)

Operator	Description	Associativity
<code>**</code>	Exponentiation	Right-to-left
<code>+</code> , <code>-</code>	Unary plus and minus	Right-to-left
<code>*</code> , <code>/</code> , <code>//</code> , <code>%</code>	Multiplication, division, integer division, and modulus	Left-to-right
<code>+</code> , <code>-</code>	Binary addition and subtraction	Left-to-right
<code><</code> , <code><=</code> , <code>></code> , <code>>=</code>	Relational operators	Left-to-right
<code>==</code> , <code>!=</code>	Equality operators	Left-to-right
<code>is</code> , <code>is not</code>	Identity operators	Left-to-right
<code>in</code> , <code>not in</code>	Membership operators	Left-to-right
<code>not</code>	Logical negation	Left-to-right
<code>and</code>	Logical conjunction	Left-to-right
<code>or</code>	Logical disjunction	Left-to-right
<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> <code>/=</code> , <code>//=</code> , <code>%=</code> , <code>**=</code>	Assignment and augmented assignment operators	Right-to-left*

List Functions/Methods

- `len(s)`
Return the number of items in a container.
- `list.append(x)`
Add an item to the end of the list. Equivalent to `a[len(a):] = [x]`.
- `list.insert(i, x)`
Insert an item at a given position. The first argument is the index of the element before which to insert.
- `list.remove(x)`
Remove the first item from the list whose value is equal to `x`. Raises a `ValueError` if no such item exists.
- `list.copy()`
Return a shallow copy of the list.
- `list.sort(key=None, reverse=False)`
Sort the items of the list in place (the arguments can be used for sort customization).
- `list.extend(iterable)`
Extend the list by appending elements from the iterable.
- `list.pop([i])`
Remove and return an item at the given position in the list. If no index is specified, `pop()` removes and returns the last item in the list.

/* Rough work */

/ Rough work */*

/ Rough work */*

/ Rough work */*